

AI ASSISTED CODING

END LAB EXAM

NAME : P.LEKHANAND

BATCH : 03

HT.NO : 2403A52044

SET : 10

A)) REFACTORING LEGACY CITY SYSTEMS

Q1: WRAP OLD SOAP SERVICE WITH MODERN REST FACADE.

• **TASK 1:** USE AI TO GENERATE WRAPPER ENDPOINTS AND TRANSFORM PAYLOADS.

PROMPT :

“CREATE REST ENDPOINTS THAT INTERNALLY CALL AN OLD SOAP SERVICE.

* CONVERT INCOMING REST JSON REQUESTS INTO SOAP XML FORMAT.

* INVOKE THE SOAP WSDL OPERATIONS.

* PARSE SOAP XML RESPONSE AND RETURN JSON.

PROVIDE CODE FOR CONTROLLERS, SERIALIZERS, AND TRANSFORMATION LOGIC.”

CODE :

```
1  const express = require("express");
2  const soap = require("soap");
3
4  const app = express();
5  app.use(express.json());
6
7  const WSDL = "https://example.com/service?wsdl"; // put your WSDL URL
8
9  app.post("/api/:op", async (req, res) => {
10    try {
11      const client = await soap.createClientAsync(WSDL);
12      const op = req.params.op; // operation name
13      const args = req.body; // JSON input
14
15      if (!client[op + "Async"])
16        return res.status(400).json({ error: "SOAP operation not found" });
17
18      const [response] = await client[op + "Async"](args);
19
20      res.json(response); // return SOAP response as JSON
21    } catch (err) {
22      res.status(500).json({ error: err.message });
23    }
24  });
25
26  app.listen(3000, () => console.log("REST -> SOAP running"));
27
```

OUTPUT :

1) CALLING GET USER

POST → HTTP://LOCALHOST:3000/API/GETUSER

BODY:

```
{ "USERID": 5 }
```

OUTPUT:

```
{
  "GETUSERRESULT": {
    "USERID": 5,
    "NAME": "JOHN DOE",
    "EMAIL": "JOHN@EXAMPLE.COM",
    "STATUS": "ACTIVE"
  }
}
```

2) CALLING CREATE USER

POST:

```
{  
  
  "NAME": "ALICE",  
  
  "EMAIL": "ALICE@TEST.COM"  
  
}
```

OUTPUT:

```
{  
  
  "CREATEUSERRESULT": {  
  
    "STATUS": "SUCCESS",  
  
    "USERID": 101  }  
  
}
```

OBSERVATION :

1. AI SCANS THE SOAP WSDL TO DETECT OPERATIONS, PARAMETERS, AND NAMESPACES.
2. IDENTIFIES WHICH REST ENDPOINTS MAP TO EACH SOAP ACTION.
3. AUTO-GENERATES JSON → SOAP XML REQUEST TRANSFORMERS.
4. BUILDS XML → JSON RESPONSE PARSERS WHILE FLATTENING NESTED SOAP NODES.
5. ENSURES SOAP ENVELOPE, HEADERS, AND TYPES REMAIN COMPLIANT.

• TASK 2: ADD TESTS AND DOCUMENTATION

PROMPT :

“CREATE UNIT TESTS FOR REST-TO-SOAP WRAPPER.

INCLUDE:

* MOCK SOAP RESPONSES

* VALIDATION OF XML GENERATION

* RESPONSE JSON FORMATTING

ALSO GENERATE DOCUMENTATION DESCRIBING EACH ENDPOINT, SAMPLE REQUESTS, ERRORS, AND TRANSFORMATION FLOW.”

CODE :

```
1 const express = require("express");
2 const app = express();
3 app.use(express.json());
4 function mockSoap(operation, args) {
5   const responses = {
6     GetUser: {
7       GetUserResult: {
8         userId: args.userId,
9         name: "John Doe",
10        email: "john@example.com",
11        status: "Active"
12      }
13    },
14    CreateUser: {
15      CreateUserResult: {
16        status: "Success",
17        userId: 101
18      }
19    }
20  };
21  return responses[operation] || { error: "SOAP operation not found" };
22 }
23 app.post("/api/:op", (req, res) => {
24   const op = req.params.op;
25   const args = req.body;
26
27   const result = mockSoap(op, args);
28
29   if (result.error) return res.status(400).json(result);
30
31   res.json(result);
32 });
33 module.exports = app;
34 if (require.main === module) {
35   app.listen(3000, () => console.log("REST + SOAP running on port 3000"));
36 }
37 if (process.env.JEST_WORKER_ID !== undefined) {
38   const request = require("supertest");
```

```
39
40 describe("REST → SOAP API TESTS", () => {
41
42   test("GetUser should return correct user", async () => {
43     const res = await request(app)
44       .post("/api/GetUser")
45       .send({ userId: 5 });
46
47     expect(res.statusCode).toBe(200);
48     expect(res.body.GetUserResult.userId).toBe(5);
49   });
50
51   test("CreateUser should return success", async () => {
52     const res = await request(app)
53       .post("/api/CreateUser")
54       .send({ name: "Alice", email: "alice@test.com" });
55
56     expect(res.statusCode).toBe(200);
57     expect(res.body.CreateUserResult.status).toBe("Success");
58   });
59
60   test("Unknown operation should return error", async () => {
61     const res = await request(app)
62       .post("/api/NoSuchOp")
63       .send({});
64
65     expect(res.statusCode).toBe(400);
66     expect(res.body.error).toBe("SOAP operation not found");
67   });
68 });
69 }
70
```

OUTPUT :

OUTPUT FOR /API/GET USER

```
{  
  
  "GetUserResult": {  
  
    "UserID": 5,  
  
    "Name": "John Doe",  
  
    "Email": "John@example.com",  
  
    "Status": "Active"  
  
  }  
}
```

OUTPUT FOR /API / CREATE USER

```
{  
  
  "CreateUserResult": {  
  
    "Status": "Success",  
  
    "UserID": 101  
  
  }  
}
```

SAMPLE ERROR OUTPUT

```
{  
  
  "Error": "SOAP OPERATION NOT FOUND"  
  
}
```

OBSERVATION :

- 1.** AI SETS UP JEST MOCKS FOR SOAP CLIENT CALLS.
- 2.** VALIDATES THAT EACH REST ENDPOINT TRIGGERS THE RIGHT SOAP ACTION.
- 3.** CONFIRMS JSON TRANSFORMATION CORRECTNESS AND ERROR HANDLING.
- 4.** GENERATES ENDPOINT DOCUMENTATION WITH INPUTS/OUTPUTS.
- 5.** ADDS EXAMPLE PAYLOADS FOR QUICK DEVELOPER REFERENCE.

B)) CONSOLIDATE DIFFERENT ALERTING FORMATS INTO UNIFIED SCHEMA.

• **TASK 1:** USE AI TO MAP AND NORMALIZE.

PROMPT :

“ANALYZE MULTIPLE ALERT FORMATS (JSON, XML, CSV) AND DERIVE A UNIFIED SCHEMA.

IDENTIFY COMMON FIELDS SUCH AS:

* ALERT ID

* SOURCE

* TIMESTAMP

* SEVERITY

* DESCRIPTION

GENERATE MAPPING RULES FOR EACH SOURCE FORMAT.”

CODE:

```
const xml2js = require("xml2js");
const csv = require("csv-parse/sync");
async function normalize(input, format) {
  if (format === "json") {
    return {
      alertId: input.id,
      source: input.source,
      timestamp: input.time,
      severity: input.level,
      description: input.msg
    };
  }

  if (format === "xml") {
    const x = await xml2js.parseStringPromise(input);
    const a = x.alert;
    return {
      alertId: a.id[0],
      source: a.source[0],
      timestamp: a.timestamp[0],
      severity: a.severity[0],
      description: a.description[0]
    };
  }

  if (format === "csv") {
    const r = csv.parse(input, { columns: true })[0];
    return {
      alertId: r.id,
      source: r.source,
      timestamp: r.timestamp,
      severity: r.severity,
      description: r.description
    };
  }
}
```

```
(async () => {
  console.log(await normalize(
    { id: "1", source: "FW", time: "T1", level: "High", msg: "Test" },
    "json"
  ));

  console.log(await normalize(
    `<alert><id>2</id><source>IDS</source><timestamp>T2</timestamp><severity>Med</severity><description>X</description></alert>`,
    "xml"
  ));

  console.log(await normalize(
    `id,source,timestamp,severity,description
3,AV,T3,Low,Blocked`,
    "csv"
  ));
})();
```

OUTPUT :

```
{  
  
  ALERTID: '1',  
  
  SOURCE: 'FW',  
  
  TIMESTAMP: 'T1',  
  
  SEVERITY: 'HIGH',  
  
  DESCRIPTION: 'TEST'  
}
```

```
{  
  
  ALERTID: '2',  
  
  SOURCE: 'IDS',  
  
  TIMESTAMP: 'T2',  
  
  SEVERITY: 'MED',  
  
  DESCRIPTION: 'X'  
}
```

```
{  
  
  ALERTID: '3',  
  
  SOURCE: 'AV',  
  
  TIMESTAMP: 'T3',  
  
  SEVERITY: 'Low',  
  
  DESCRIPTION: 'BLOCKED'  
}
```


OBSERVATION :

1. AI ANALYSES XML, CSV, JSON ALERT SAMPLES TO FIND COMMON FIELDS.
2. DETECTS NAMING CONFLICTS AND RESOLVES THEM TO ONE STANDARD KEY SET.
3. NORMALIZES TIMESTAMP FORMATS INTO ISO 8601.
4. DEFINES UNIFIED SCHEMA WITH CONSISTENT SEVERITY, SOURCE, AND METADATA.
5. GENERATES MAPPING RULES FROM EACH ORIGINAL FORMAT → UNIFIED SCHEMA.

• **TASK 2:** WRITE ADAPTOR LIBRARY AND TESTS.

PROMPT :

“IMPLEMENT ADAPTORS THAT CONVERT XML/JSON/CSV ALERT FORMATS INTO THE UNIFIED ALERT SCHEMA.

PROVIDE:

* PARSER FUNCTIONS

* VALIDATION LOGIC

* ERROR HANDLING

ALSO GENERATE UNIT TESTS FOR EACH INPUT FORMAT.”

CODE :

```

1  const xml2js = require("xml2js");
2  const csv = require("csv-parse/sync");
3
4  const ok = a => ([ "alertId","source","timestamp","severity","description"
5    ].forEach(f => { if (!a[f]) throw f; }), a);
6
7  const fromJSON = j => ok({
8    alertId:j.id, source:j.source, timestamp:j.time,
9    severity:j.level, description:j.msg
10   });
11
12  const fromXML = async x => {
13    const a = (await xml2js.parseStringPromise(x)).alert;
14    return ok({
15      alertId:a.id[0], source:a.source[0], timestamp:a.timestamp[0],
16      severity:a.severity[0], description:a.description[0]
17    });
18  };
19
20  const fromCSV = c => {
21    const r = csv.parse(c, {columns:true})[0];
22    return ok({
23      alertId:r.id, source:r.source, timestamp:r.timestamp,
24      severity:r.severity, description:r.description
25    });
26  };
27  (async ()=>{
28    console.log(fromJSON({id:"1",source:"FW",time:"T1",level:"High",msg:"X"}));
29    console.log(await fromXML(`<alert><id>2</id><source>IDS</source><timestamp>T2</timestamp><severity>M</severity><description>Y</description></alert>`));
30    console.log(fromCSV(`id,source,timestamp,severity,description\n3,AV,T3,Low,Z`));
31  })();
32

```

OUTPUT :

```
{
```

ALERTID: '1',

SOURCE: 'FW',

TIMESTAMP: 'T1',

SEVERITY: 'HIGH',

DESCRIPTION: 'X'

```
}
```

```
{
```

ALERTID: '2',

SOURCE: 'IDS',

TIMESTAMP: 'T2',

SEVERITY: 'M',

DESCRIPTION: 'Y'

```
}
```

```
{
```

```
  ALERTID: '3',
```

```
  SOURCE: 'AV',
```

```
  TIMESTAMP: 'T3',
```

```
  SEVERITY: 'Low',
```

```
  DESCRIPTION: 'Z'
```

```
}
```

OBSERVATION :

1. AI BUILDS MODULAR ADAPTORS (XML/CSV/JSON) USING PARSING LIBRARIES.
2. APPLIES MAPPING RULES TO NORMALIZE ALL INPUTS.
3. VALIDATES CONVERSIONS AGAINST THE UNIFIED SCHEMA.
4. CREATES JEST TESTS WITH MULTIPLE INPUT SAMPLES.
5. TESTS ERROR HANDLING FOR MALFORMED OR MISSING FIELDS.